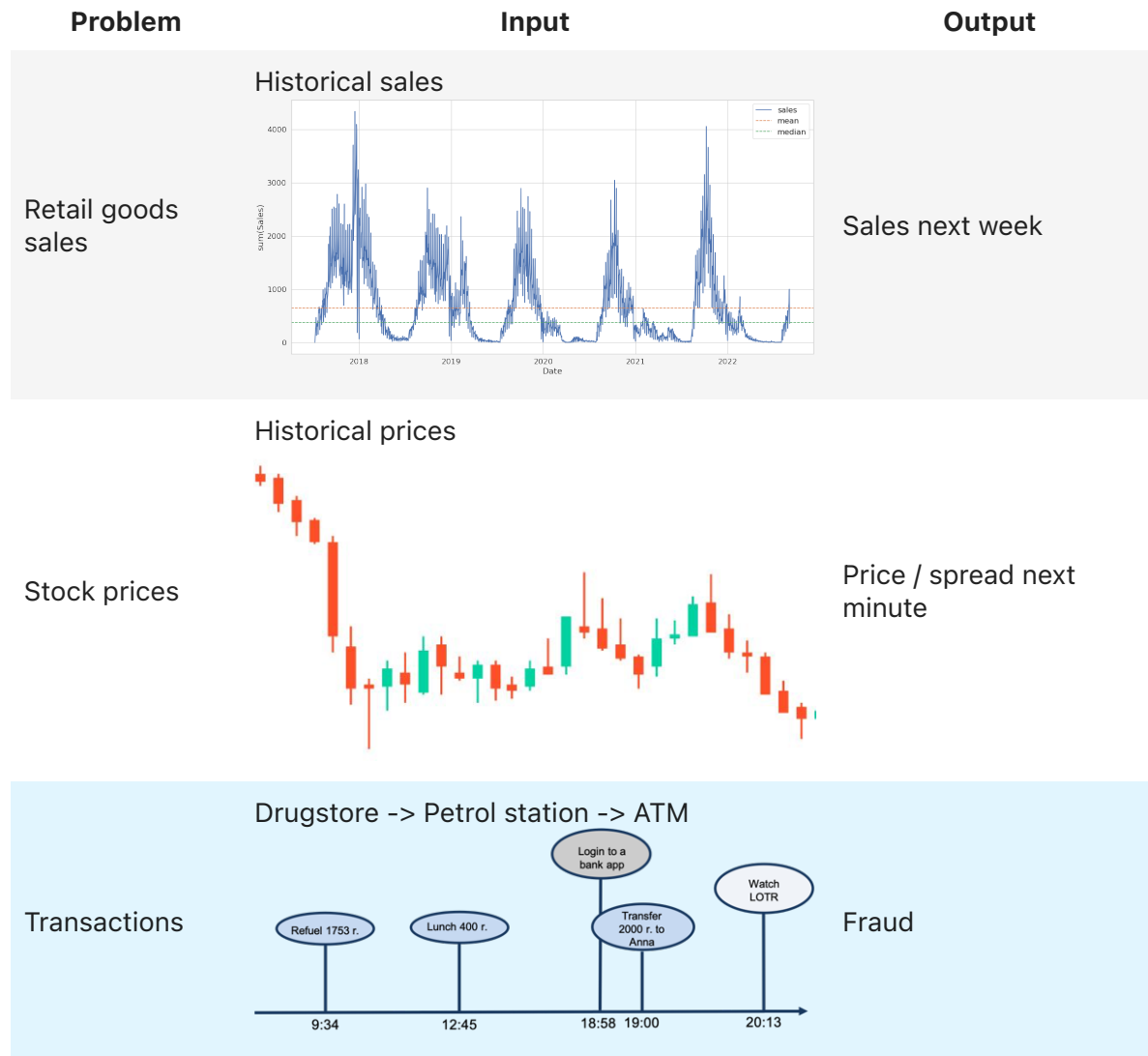


Lecture

Machine Learning problems with sequential data



Deep Learning problems with sequential data

Problem	Input	Output
Speech recognition	Speech	Text
Sentiment classification	Text	Label
Machine translation	Text in one language	Text in other language
Video activity recognition	Video	Label

Note the differences in input/output size and relations

ML regression

feature 1	feature 2	feature 3	...	feature k	target
x_{11}	x_{12}	x_{13}	...	x_{1k}	y_1
x_{21}	x_{22}	x_{23}	...	x_{2k}	y_2
x_{31}	x_{32}	x_{33}	...	x_{3k}	y_3
...	...	...	...	...	...
x_{n1}	x_{n2}	x_{n3}	...	x_{nk}	y_n

- Regularized linear regression

$$\arg \min_{\theta} \|y - X\theta\|^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2$$

- Gradient boosting

$$F_{m+1}(x_i) = F_m(x_i) + h_m(x_i) \approx y_i$$

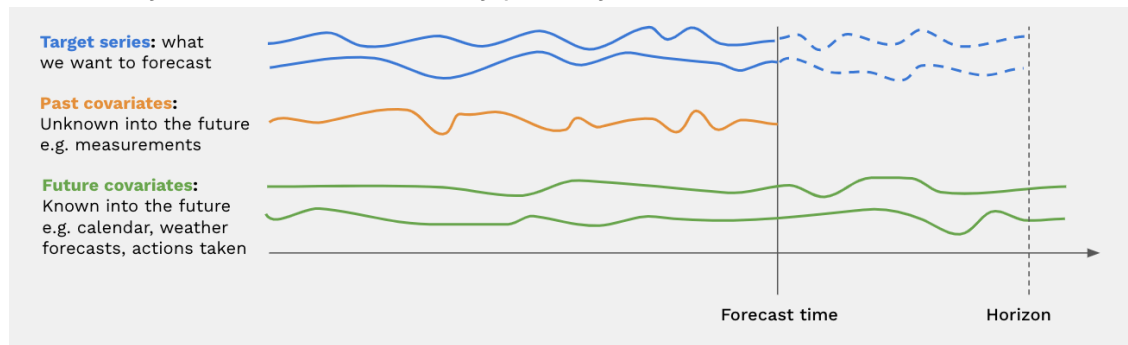
ML regression for time series forecasting

feature 1	feature 2	feature 3	...	feature k	target
x_{11}	x_{12}	x_{13}	...	x_{1k}	y_1
x_{21}	x_{22}	x_{23}	...	x_{2k}	y_2
x_{31}	x_{32}	x_{33}	...	x_{3k}	y_3
...	...	...	...	...	...
x_{n1}	x_{n2}	x_{n3}	...	x_{nk}	y_n

- Take the series history $y_t, y_{t-1}, y_{t-2}, \dots$
- Add external dynamic data $x_{t1}, x_{t2}, \dots, x_{(t-1)1}, x_{(t-1)2}, \dots$
- Add features like differences, combinations, rolling means, medians, stds, etc
- Add encoded categorical features like day of week, holidays
- Add static features like classes
- Apply regression to y_{t+1}

ML regression for time series forecasting

- External dynamic features was only partially available in classic time series



- External static features was not available in classic time series

```
In [1]: import numpy as np
import pandas as pd

import IPython.display as dp
import matplotlib.pyplot as plt
import seaborn as sns

dp.set_matplotlib_formats("retina")
sns.set(style="whitegrid", font_scale=1.5)
sns.despine()

%matplotlib inline
```

```
/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_87659/321256244
3.py:8: DeprecationWarning: `set_matplotlib_formats` is deprecated since I
Python 7.23, directly use `matplotlib_inline.backend_inline.set_matplotlib
_formats()`
  dp.set_matplotlib_formats("retina")
<Figure size 640x480 with 0 Axes>
```

```
In [2]: from darts import TimeSeries
import darts.datasets as ds
import scipy.stats as sts
import statsmodels.api as sm
import statsmodels.tsa.api as tsa
from darts.utils.statistics import plot_acf
```

```
In [3]: from darts import concatenate
```

```
In [4]: from astsadata import cmort as cmort_pd, part as part_pd, tempr as tempr_
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:15: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
  birth = pd.read_csv(f"{path}/data/birth.csv", index_col=0, parse_dates=T
rue)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:27: UserWarning: Could not infer format, so each element wi
```

```
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
cardox = pd.read_csv(f"{path}/data/cardox.csv", index_col=0, parse_dates
=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:30: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
chicken = pd.read_csv(f"{path}/data/chicken.csv", index_col=0, parse_dat
es=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:36: UserWarning: Converting to PeriodArray/Index representa
tion will drop timezone information.
cmort.index = cmort.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:54: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
flu = pd.read_csv(f"{path}/data/flu.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:61: UserWarning: Converting to PeriodArray/Index representa
tion will drop timezone information.
gas.index = gas.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:63: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
gdp = pd.read_csv(f"{path}/data/gdp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:72: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
gnp = pd.read_csv(f"{path}/data/gnp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:90: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
hor = pd.read_csv(f"{path}/data/hor.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:93: UserWarning: Could not infer format, so each element wi
ll be parsed individually, falling back to `dateutil`. To ensure parsing i
s consistent and as-expected, please specify a format.
jj = pd.read_csv(f"{path}/data/jj.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:97: UserWarning: Converting to PeriodArray/Index representa
tion will drop timezone information.
lap.index = lap.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:109: UserWarning: Converting to PeriodArray/Index represent
ation will drop timezone information.
oil.index = oil.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadat
a/astsadata.py:112: UserWarning: Converting to PeriodArray/Index represent
ation will drop timezone information.
```

```
part.index = part.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:117: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    polio = pd.read_csv(f"{path}/data/polio.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:120: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    prodn = pd.read_csv(f"{path}/data/prodn.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:123: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    qinfl = pd.read_csv(f"{path}/data/qinfl.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:126: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    qintr = pd.read_csv(f"{path}/data/qintr.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:129: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    rec = pd.read_csv(f"{path}/data/rec.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:135: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    salmon = pd.read_csv(f"{path}/data/salmon.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:145: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
    so2.index = so2.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:147: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    soi = pd.read_csv(f"{path}/data/soi.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:158: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    sunspotz = pd.read_csv(f"{path}/data/sunspotz.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsa/astsa.py:162: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
```

```

    tempr.index = tempr.index.to_period("W")
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:164: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    unemp = pd.read_csv(f"{path}/data/unemp.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:167: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
    UnempRate = pd.read_csv(f"{path}/data/UnempRate.csv", index_col=0, parse_dates=True)
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:176: FutureWarning: The argument 'date_parser' is deprecated and will be removed in a future version. Please use 'date_format' instead, or read your data in as 'object' dtype and then call 'to_datetime'.
    djia = pd.read_csv(
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:180: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings is deprecated. In a future version, strings will be parsed as datetime strings, matching the behavior without a 'unit'. To retain the old behavior, explicitly cast ints or floats to numeric type before calling to_datetime.
    date_parser=lambda x: pd.to_datetime(x, unit="s"),
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/astsadata/astsadata.py:182: FutureWarning: PeriodDtype[B] is deprecated and will be removed in a future version. Use a DatetimeIndex with freq='B' instead
    djia.index = djia.index.to_period("B")

```

```

In [5]: cmort = TimeSeries.from_series(
    cmort_pd["value"].rename("mort").set_axis(
        pd.date_range(
            cmort_pd.index[0].start_time.date(),
            periods=len(cmort_pd),
            freq="W-MON",
            inclusive="both"
        )
    ).resample("M").mean()
)
part = TimeSeries.from_series(
    part_pd["value"].rename("part").set_axis(
        pd.date_range(
            part_pd.index[0].start_time.date(),
            periods=len(part_pd),
            freq="W-MON",
            inclusive="both"
        )
    ).resample("M").mean()
)
tempr = TimeSeries.from_series(
    tempr_pd["value"].rename("temp").set_axis(
        pd.date_range(
            tempr_pd.index[0].start_time.date(),

```

```

        periods=len(tempr_pd),
        freq="W-MON",
        inclusive="both"
    )
    ).resample("M").mean()
)

```

```

In [6]: cmort_train, cmort_test = cmort.split_before(pd.Timestamp("1978-01-01"))
part_cov, _ = part.split_before(pd.Timestamp("1978-01-01"))
temp_cov = tempr

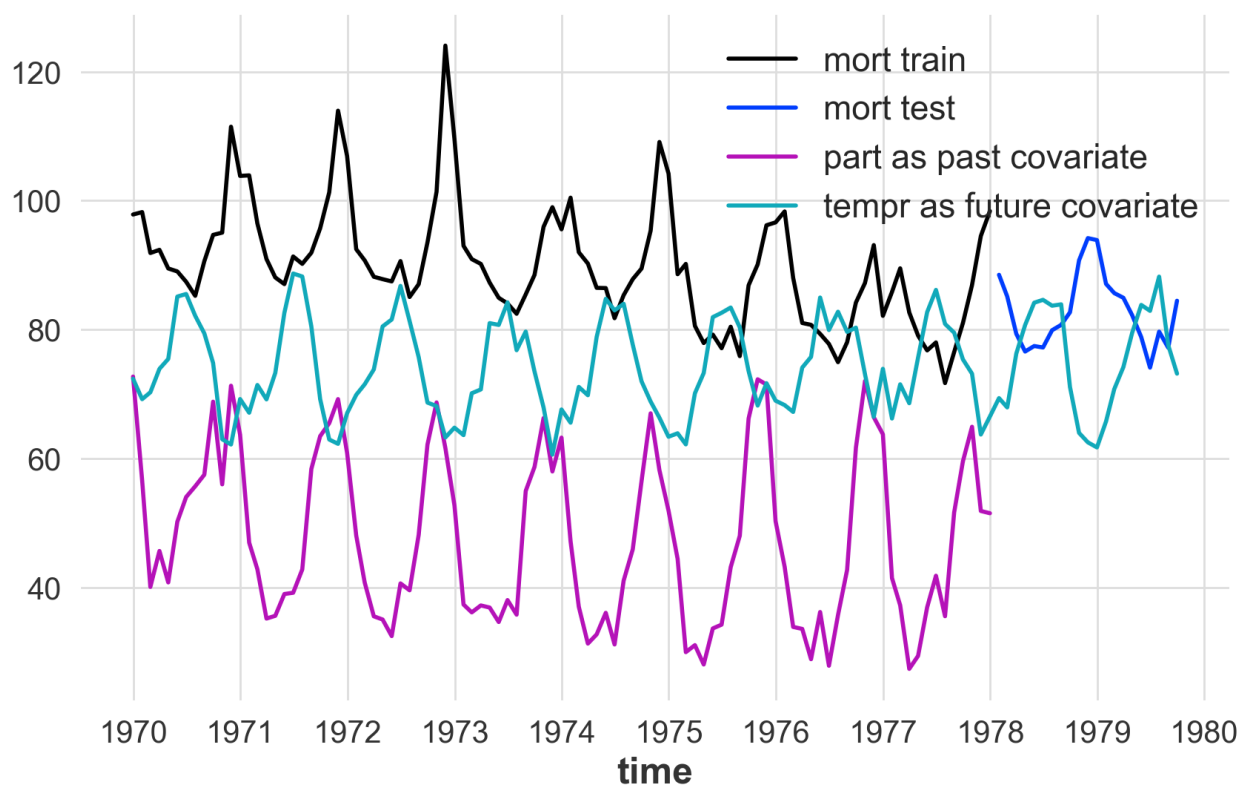
```

```

In [7]: fig, ax = plt.subplots(figsize=(10,6))
cmort_train.plot(ax=ax, label="mort train")
cmort_test.plot(ax=ax, label="mort test")
part_cov.plot(ax=ax, label="part as past covariate")
temp_cov.plot(ax=ax, label="tempr as future covariate")

```

Out[7]: <Axes: xlabel='time'>



```

In [8]: from darts.models import LinearRegressionModel, RegressionModel, RandomFo
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/tqdm/aut
o.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywi
dgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

```

```

In [9]: from darts.metrics import mape

```

```

In [10]: model = LinearRegressionModel(
        lags=[-1, -2, -3, -4, -5, -6],

```

```

        output_chunk_length=20,
    )
    model.fit(cmort_train);
    pred = model.predict(20)

```

```

In [11]: model_past = LinearRegressionModel(
        lags=[-1, -2, -3, -4, -5, -6],
        lags_past_covariates=[-1, -2, -3, -4, -5, -6],
        output_chunk_length=20,
    )
    model_past.fit(cmort_train, past_covariates=part_cov);
    pred_past = model_past.predict(20)

```

```

In [12]: model_future = LinearRegressionModel(
        lags=[-1, -2, -3, -4, -5, -6],
        lags_future_covariates=[0, -1, -2, -3, -4, -5, -6],
        output_chunk_length=20,
    )
    model_future.fit(cmort_train, future_covariates=temp_cov);
    pred_future = model_future.predict(20)

```

```

In [13]: model_past_future = LinearRegressionModel(
        lags=[-1, -2, -3, -4, -5, -6],
        lags_past_covariates=[-1, -2, -3, -4, -5, -6],
        lags_future_covariates=[0, -1, -2, -3, -4, -5, -6],
        output_chunk_length=20,
    )
    model_past_future.fit(cmort_train, past_covariates=part_cov, future_covariates=temp_cov);
    pred_past_future = model_past_future.predict(20)

```

```

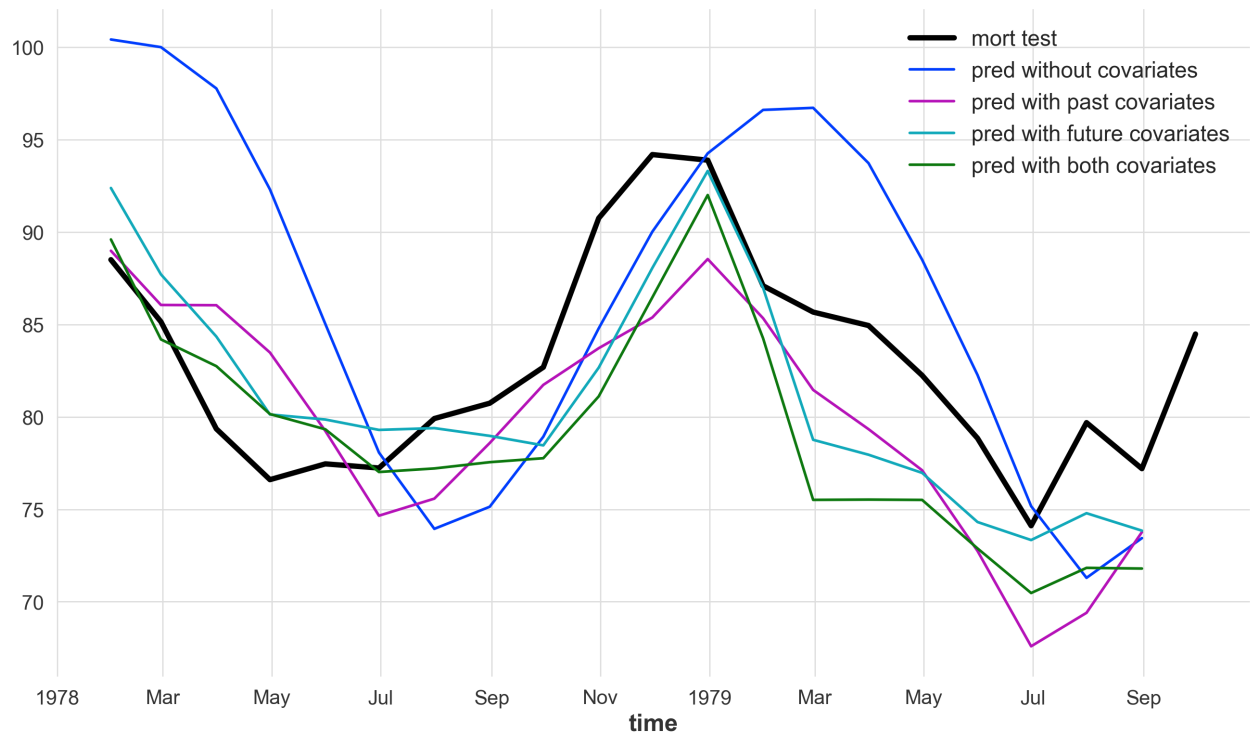
In [14]: fig, ax = plt.subplots(figsize=(16,9))
    cmort_test.plot(ax=ax, lw=4, label="mort test")
    pred.plot(ax=ax, label="pred without covariates")
    pred_past.plot(ax=ax, label="pred with past covariates")
    pred_future.plot(ax=ax, label="pred with future covariates")
    pred_past_future.plot(ax=ax, label="pred with both covariates")

```

```

Out[14]: <Axes: xlabel='time'>

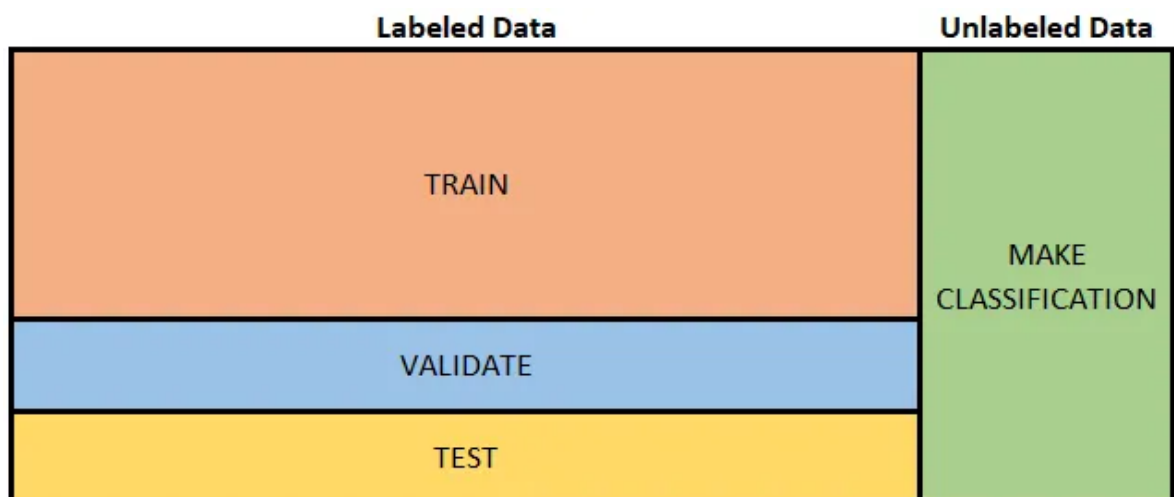
```



```
In [15]: mape(
    [cmort_test, cmort_test, cmort_test, cmort_test],
    [pred, pred_past, pred_future, pred_past_future]
)
```

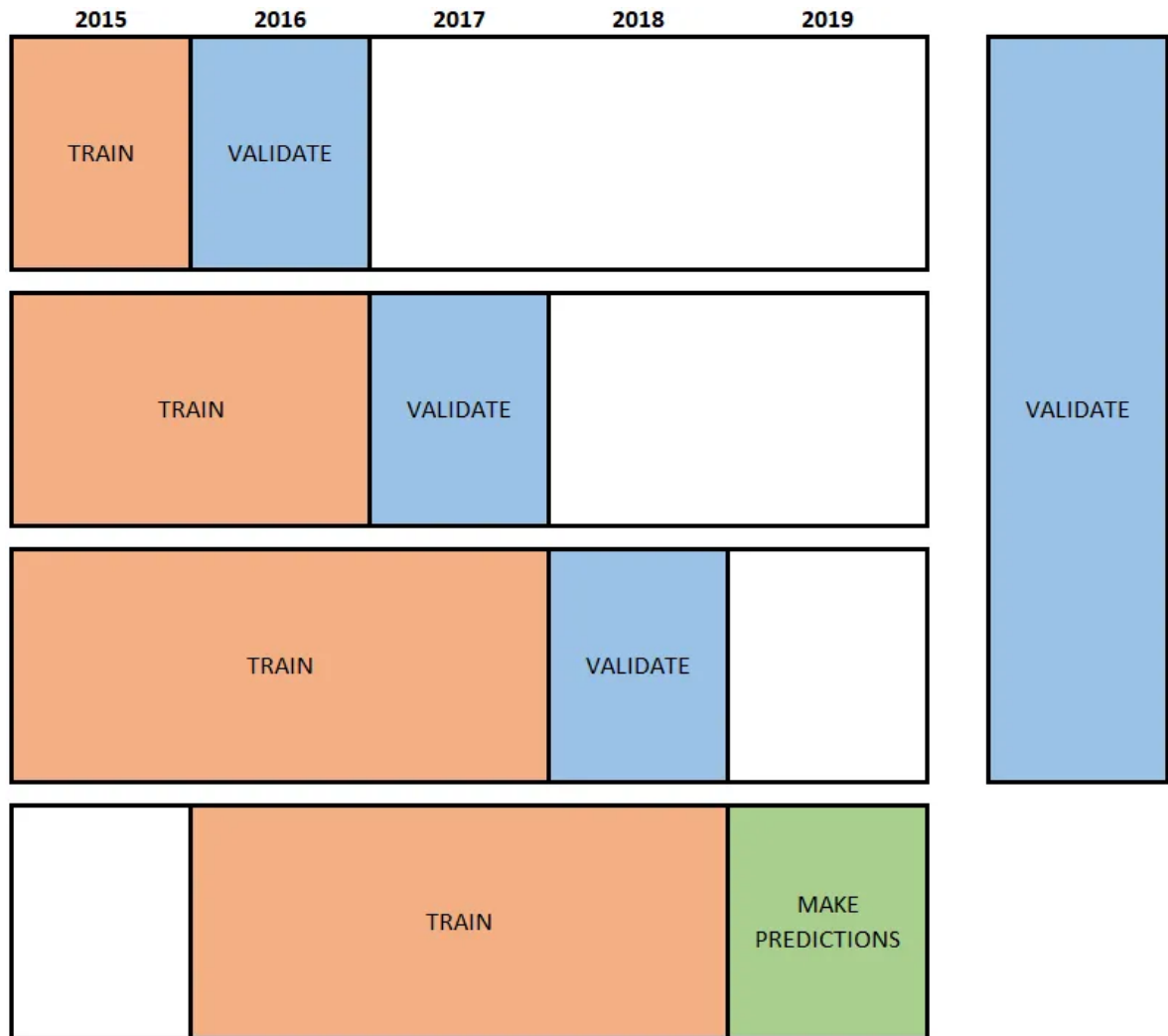
```
Out[15]: [8.937341507111249, 5.518518759087556, 4.408916516691464, 5.590476275937467]
```

Cross-validation



- Pro: trains using latest data
- Con: no idea how model performs out-of-time

Cross-validation of time series (out-of-time)



- Pro: model building takes into account ability to predict out-of-time
- Con: validation set is not random

```
In [16]: model = LinearRegressionModel(
    lags=[-1, -2, -3, -4, -5, -6],
    lags_future_covariates=[0, -1, -2, -3, -4, -5, -6],
)
historical_forecasts = model.historical_forecasts(
    cmort_train, future_covariates=temp_cov, train_length=36, forecast_ho
);
```

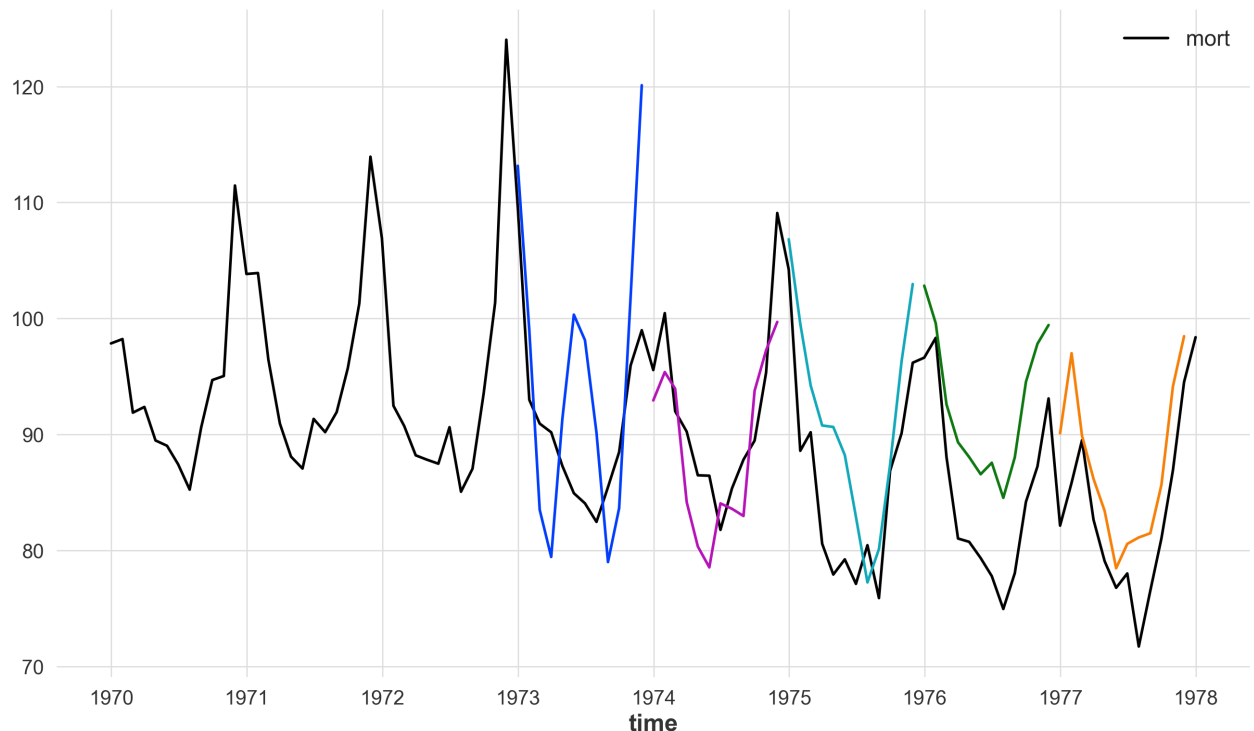
`enable\_optimization=True` is ignored because `retrain` is not `False` or `0`. To hide this warning, set `show\_warnings=False` or `enable\_optimization=False`.

`enable\_optimization=True` is ignored because `forecast\_horizon > model.output\_chunk\_length`. To hide this warning, set `show\_warnings=False` or `enable\_optimization=False`.

```
In [17]: [f"{forecast.start_time()} - {forecast.end_time()}", {len(forecast)} value
```

```
Out[17]: ['1972-12-31 00:00:00 - 1973-11-30 00:00:00, 12 values',
          '1973-12-31 00:00:00 - 1974-11-30 00:00:00, 12 values',
          '1974-12-31 00:00:00 - 1975-11-30 00:00:00, 12 values',
          '1975-12-31 00:00:00 - 1976-11-30 00:00:00, 12 values',
          '1976-12-31 00:00:00 - 1977-11-30 00:00:00, 12 values']
```

```
In [88]: fig, ax = plt.subplots(figsize=(16,9))
cmort_train.plot(ax=ax)
[forecast.plot(label=None, ax=ax) for forecast in historical_forecasts];
```



```
In [89]: model = LinearRegressionModel(
    lags=[-1, -2, -3, -4, -5, -6],
    lags_future_covariates=[0, -1, -2, -3, -4, -5, -6],
)
historical_forecasts = model.historical_forecasts(
    cmort_train, future_covariates=temp_cov, train_length=36, forecast_ho
```

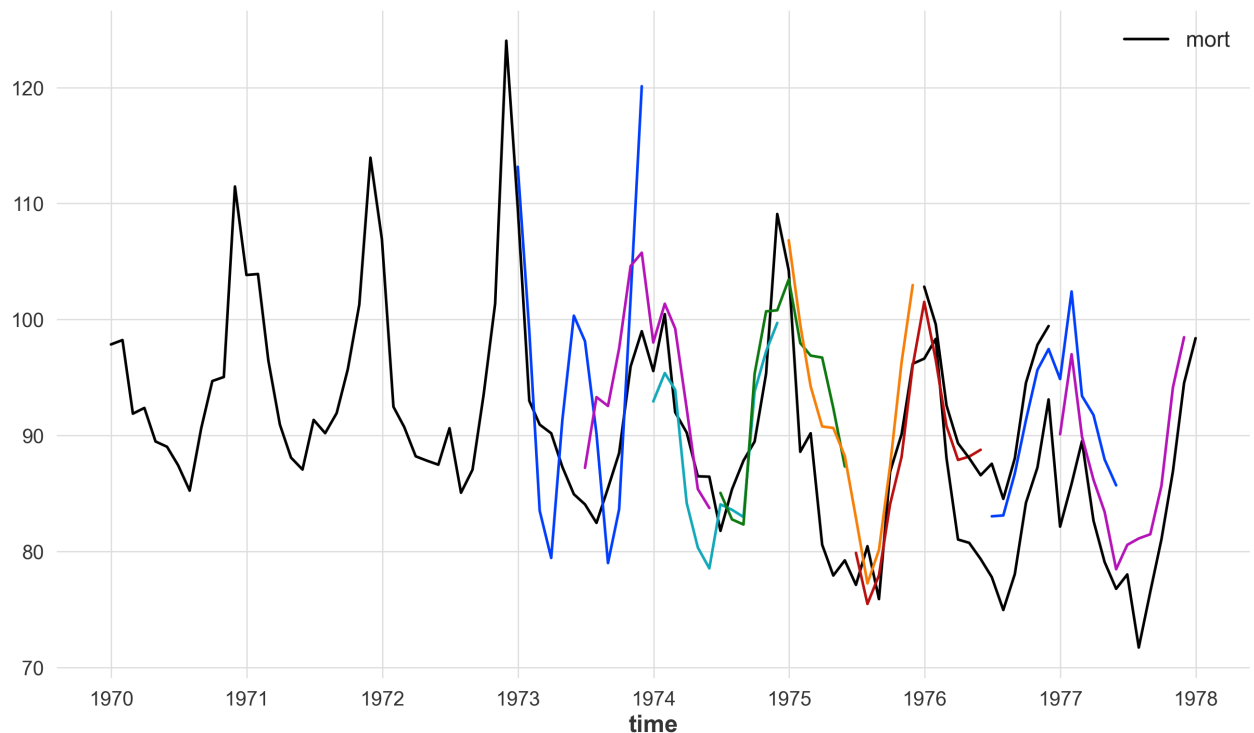
`enable\_optimization=True` is ignored because `retrain` is not `False` or `0`. To hide this warning, set `show\_warnings=False` or `enable\_optimization=False`.

`enable\_optimization=True` is ignored because `forecast\_horizon > model.output\_chunk\_length`. To hide this warning, set `show\_warnings=False` or `enable\_optimization=False`.

```
In [90]: [f"{forecast.start_time()} - {forecast.end_time()}, {len(forecast)} value
```

```
Out[90]: ['1972-12-31 00:00:00 - 1973-11-30 00:00:00, 12 values',
'1973-06-30 00:00:00 - 1974-05-31 00:00:00, 12 values',
'1973-12-31 00:00:00 - 1974-11-30 00:00:00, 12 values',
'1974-06-30 00:00:00 - 1975-05-31 00:00:00, 12 values',
'1974-12-31 00:00:00 - 1975-11-30 00:00:00, 12 values',
'1975-06-30 00:00:00 - 1976-05-31 00:00:00, 12 values',
'1975-12-31 00:00:00 - 1976-11-30 00:00:00, 12 values',
'1976-06-30 00:00:00 - 1977-05-31 00:00:00, 12 values',
'1976-12-31 00:00:00 - 1977-11-30 00:00:00, 12 values']
```

```
In [91]: fig, ax = plt.subplots(figsize=(16,9))
cmort_train.plot(ax=ax)
[forecast.plot(label=None, ax=ax) for forecast in historical_forecasts];
```



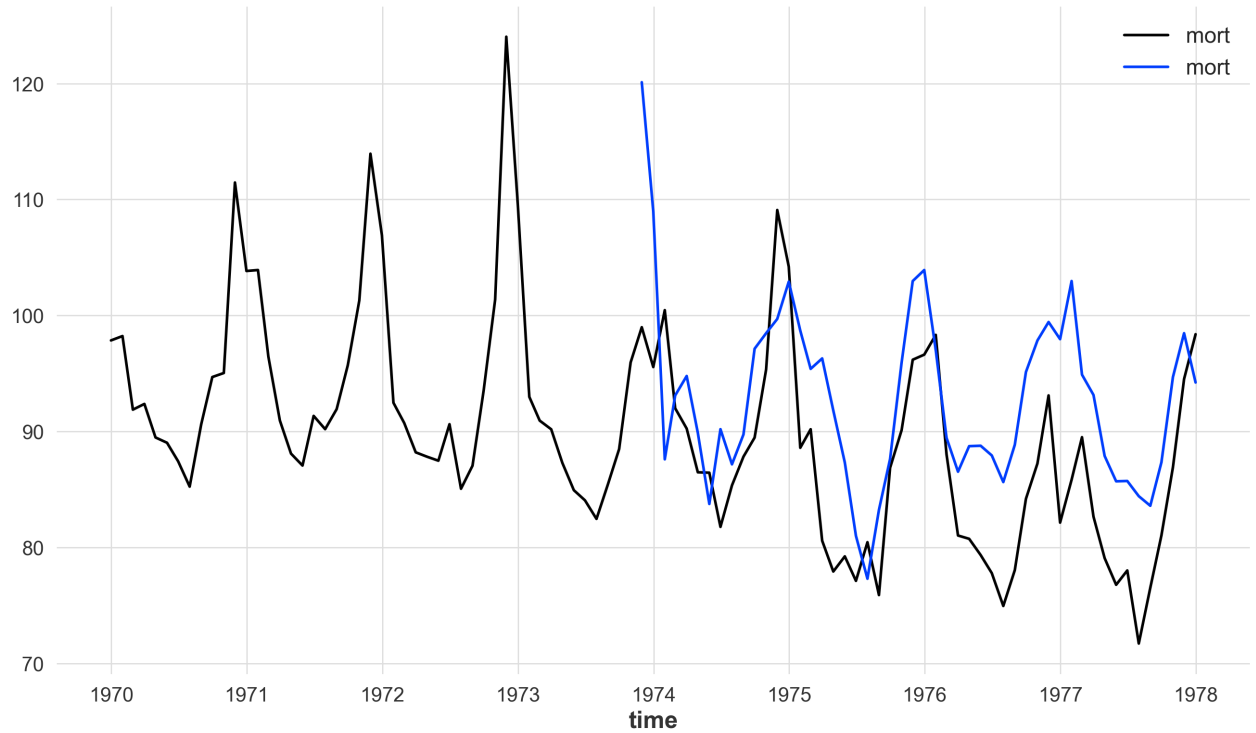
```
In [18]: model = LinearRegressionModel(
    lags=[-1, -2, -3, -4, -5, -6],
    lags_future_covariates=[0, -1, -2, -3, -4, -5, -6],
)
historical_forecasts = model.historical_forecasts(
    cmort_train, future_covariates=temp_cov, train_length=36, forecast_ho
);
```

`enable\_optimization=True` is ignored because `retrain` is not `False` or `0`. To hide this warning, set `show\_warnings=False` or `enable\_optimization=False`.

`enable\_optimization=True` is ignored because `forecast\_horizon > model.output\_chunk\_length`. To hide this warning, set `show\_warnings=False` or `enable\_optimization=False`.

```
In [20]: fig, ax = plt.subplots(figsize=(16,9))
cmort_train.plot(ax=ax)
historical_forecasts.plot(ax=ax)
```

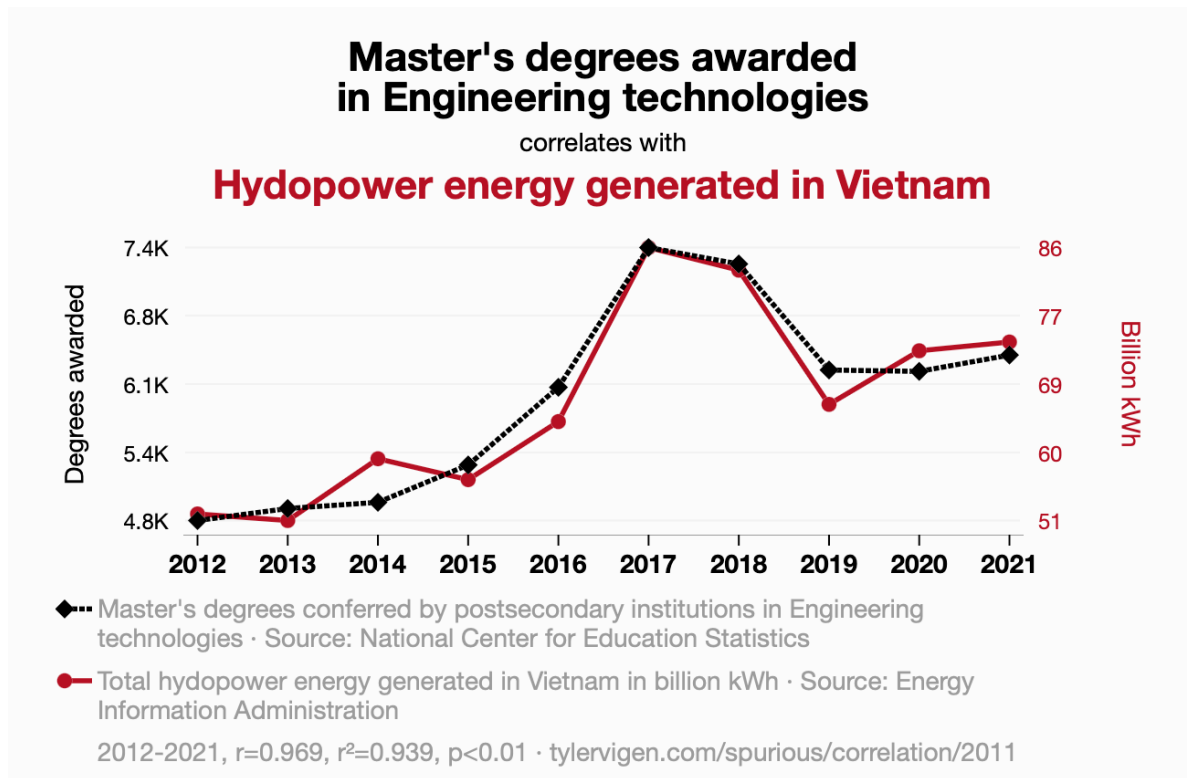
```
Out[20]: <Axes: xlabel='time'>
```



Features for time series

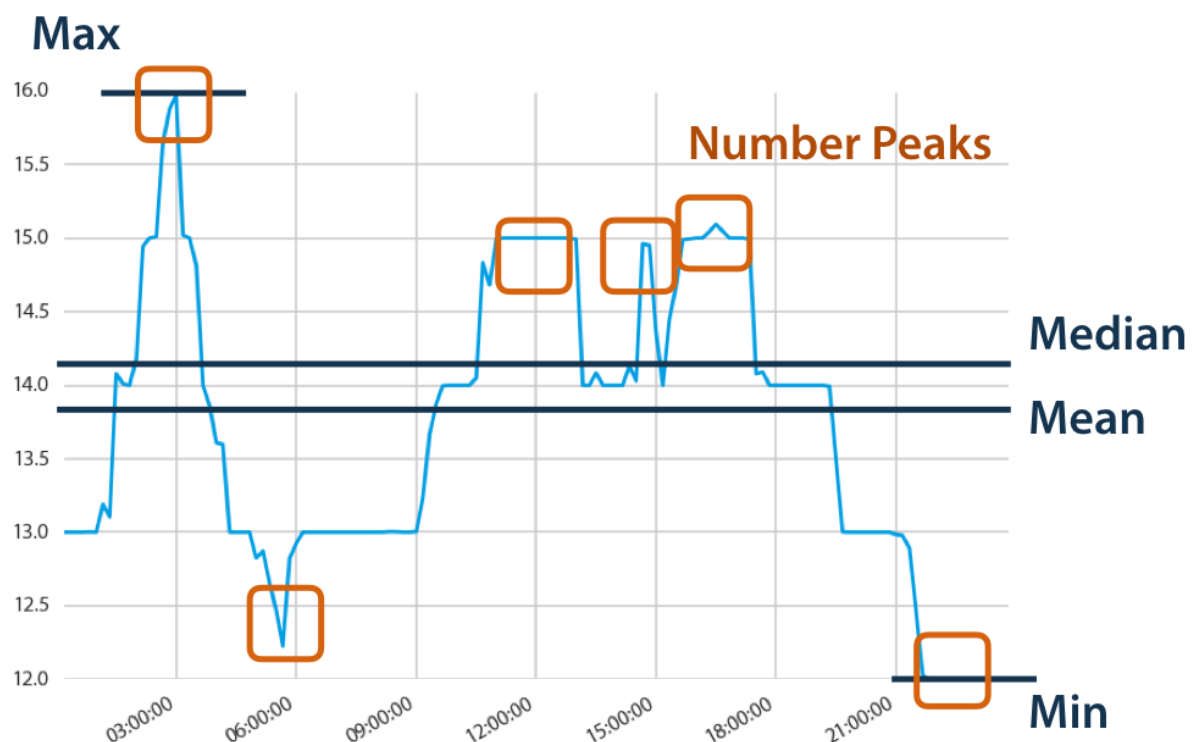
Where to take covariates from? It would be nice to have a storage of all time series in the world and always use everything for predictions... but

- Too much features
- [Spurious correlations](#)
- Hard to collect



Features for time series

Where to take covariates from? **tsfresh** is a package that does automatic extraction of up to 1558 features, provides a feature selection procedure.



```
In [93]: !pip install tsfresh
```

Collecting tsfresh

Downloading tsfresh-0.20.2-py2.py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: requests>=2.9.1 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (2.31.0)
Requirement already satisfied: numpy>=1.15.1 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (1.26.2)
Requirement already satisfied: pandas>=0.25.0 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (2.1.3)
Requirement already satisfied: scipy>=1.2.0 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (1.11.4)
Requirement already satisfied: statsmodels>=0.13 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (0.14.0)
Requirement already satisfied: patsy>=0.4.1 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (0.5.4)
Requirement already satisfied: scikit-learn>=0.22.0 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (1.3.2)
Requirement already satisfied: tqdm>=4.10.0 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (4.66.2)
Collecting stumpy>=1.7.2 (from tsfresh)

Downloading stumpy-1.12.0-py3-none-any.whl.metadata (27 kB)
Requirement already satisfied: cloudpickle in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from tsfresh) (3.0.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from pandas>=0.25.0->tsfresh) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from pandas>=0.25.0->tsfresh) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.1 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from pandas>=0.25.0->tsfresh) (2023.3)
Requirement already satisfied: six in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from patsy>=0.4.1->tsfresh) (1.16.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from requests>=2.9.1->tsfresh) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from requests>=2.9.1->tsfresh) (3.6)
Requirement already satisfied: urllib3<3,>=1.21.1 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from requests>=2.9.1->tsfresh) (2.1.0)
Requirement already satisfied: certifi>=2017.4.17 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from requests>=2.9.1->tsfresh) (2023.11.17)
Requirement already satisfied: joblib>=1.1.1 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from scikit-learn>=0.22.0->tsfresh) (1.3.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from scikit-learn>=0.22.0->tsfresh) (3.2.0)
Requirement already satisfied: packaging>=21.3 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from statsmodels>=0.13->tsfresh) (23.2)

Requirement already satisfied: numba>=0.55.2 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from stumpy>=1.7.2->tsfresh) (0.59.0)

Requirement already satisfied: llvmlite<0.43,>=0.42.0dev0 in /Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages (from numba>=0.55.2->stumpy>=1.7.2->tsfresh) (0.42.0)

Downloading tsfresh-0.20.2-py2.py3-none-any.whl (95 kB)

95.8/95.8 kB 1.1 MB/s eta 0:00:00a 0:00:01

Downloading stumpy-1.12.0-py3-none-any.whl (169 kB)

169.1/169.1 kB 1.9 MB/s eta 0:00:00a 0:00:01

Installing collected packages: stumpy, tsfresh

Successfully installed stumpy-1.12.0 tsfresh-0.20.2

```
In [95]: from tsfresh.feature_extraction import extract_features
        from tsfresh.feature_selection import select_features
```

```
In [105... data = pd.concat([forecast.pd_dataframe() for forecast in historical_fore
```

```
In [107... data["id"] = [1] * 12 + [2] * 12 + [3] * 12 + [4] * 12 + [5] * 12 + [6] *
        data = data.reset_index()
```

```
In [108... features = extract_features(data, column_id="id", column_sort="time")
```

Feature Extraction: 100%|██████████| 9/9 [00:01<00:00, 4.8 6it/s]

```
In [109... features.shape
```

```
Out[109... (9, 783)
```

```
In [110... features.columns
```

```
Out[110... Index(['mort__variance_larger_than_standard_deviation',
        'mort__has_duplicate_max', 'mort__has_duplicate_min',
        'mort__has_duplicate', 'mort__sum_values', 'mort__abs_energy',
        'mort__mean_abs_change', 'mort__mean_change',
        'mort__mean_second_derivative_central', 'mort__median',
        ...,
        'mort__fourier_entropy__bins_5', 'mort__fourier_entropy__bins_1
0',
        'mort__fourier_entropy__bins_100',
        'mort__permutation_entropy__dimension_3__tau_1',
        'mort__permutation_entropy__dimension_4__tau_1',
        'mort__permutation_entropy__dimension_5__tau_1',
        'mort__permutation_entropy__dimension_6__tau_1',
        'mort__permutation_entropy__dimension_7__tau_1',
        'mort__query_similarity_count__query_None__threshold_0.0',
        'mort__mean_n_absolute_max__number_of_maxima_7'],
        dtype='object', length=783)
```

```
In [111... features
```

Out [111]...

	mort__variance_larger_than_standard_deviation	mort__has_duplicate_max	mort.
1	1.0	0.0	
2	1.0	0.0	
3	1.0	0.0	
4	1.0	0.0	
5	1.0	0.0	
6	1.0	0.0	
7	1.0	0.0	
8	1.0	0.0	
9	1.0	0.0	

9 rows x 783 columns

Time series classification

- [Challenge](#)
- Goal: classify curves into classes
- Goal: unified approach for all problems, a one model to rule them all

	Filter	Value	Value	Value	Value
Train size	Less than 100	100 to 500	Greater than 500		
Test size	Less than 300	300 to 1000	Greater than 1000		
Length	Less than 300	300 to 700	Greater than 700		
Classes	Less than 10	10 to 30	Greater than 30		
Type	Motion	Image	Sensor	...	

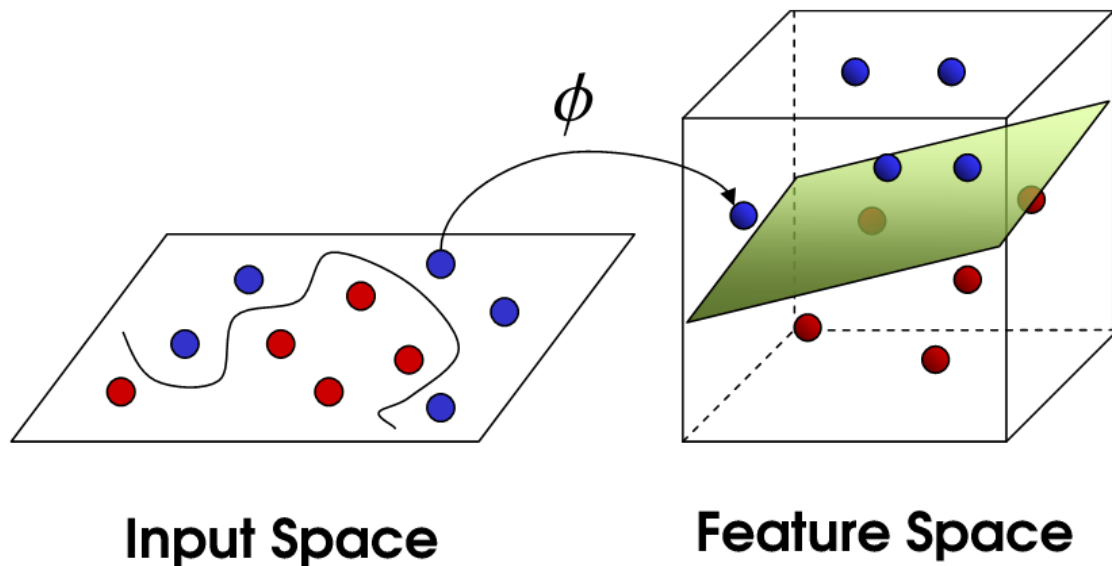
Time series classification

Challenge results:

- Approaches are heuristical in nature and consider various ideas, as time series are diverse
- Ensembles of ideas work very well and do in fact "rule them all"
- "Representations" are comparable and are faster

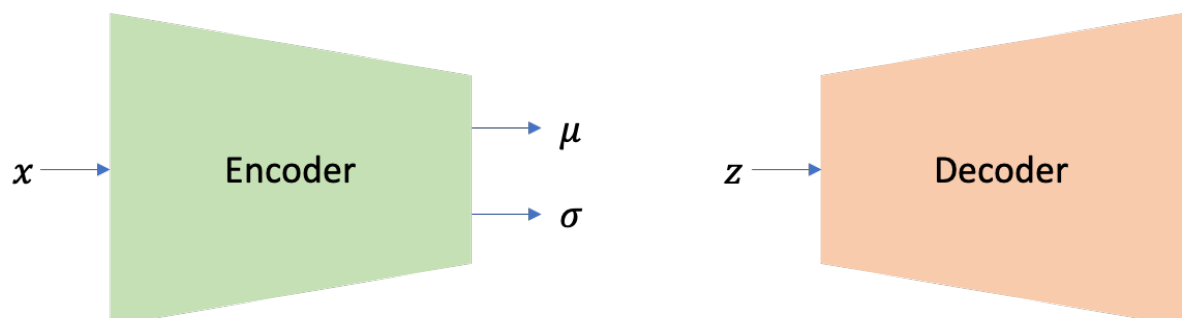
Representation learning

Features from e.g. tsfresh are very informative and rich, but they are not suited for ML models, or the ML models are not well suited to draw decision boundaries in such feature spaces. The ML models work best when the space is e.g. linearly separable, thus kernel trick in SVM.

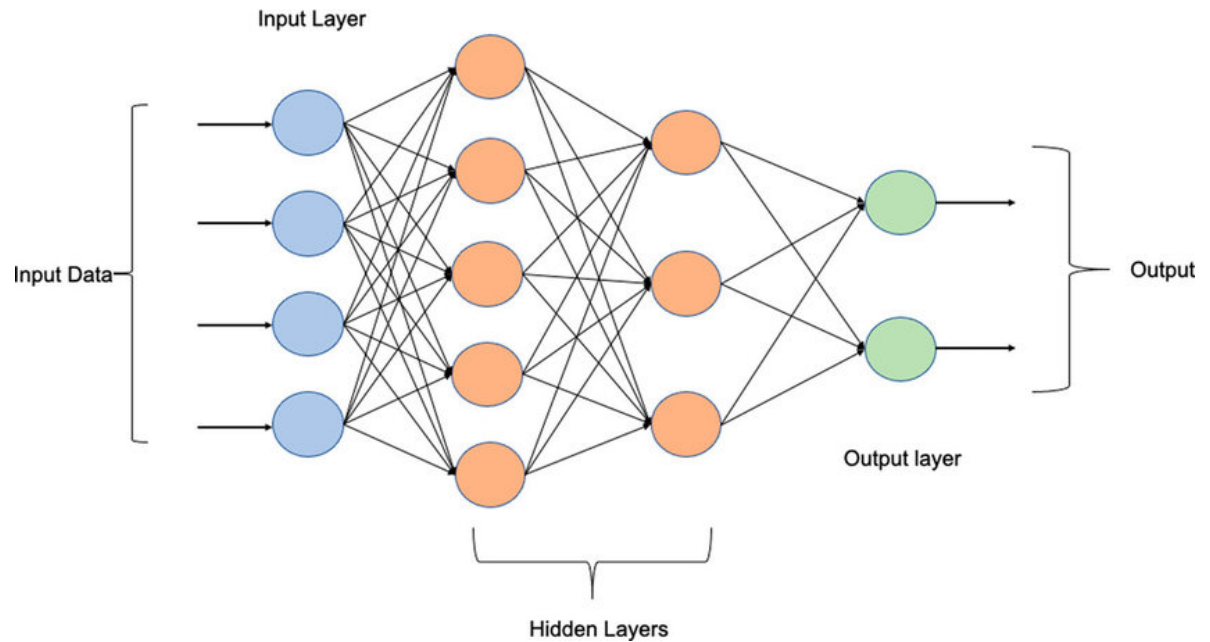


Kernel trick uses pre-defined transformations, but what if we learn optimal transformations from data? This would be representation learning, learning the optimal transformation of input features for a given task.s

Encoder-decoder architecture



Encoder-decoder architecture



Webinar

In [17]: `import pickle`

In [18]: `with open("returns_NASDAQ_hist_daily_06102021-091329.pkl", "rb") as f:
data = pickle.load(f)`

In [19]: `data.head()`

Out[19]:

	Ticker	Close
Time		
1999-11-01	ATVI	NaN
1999-11-02	ATVI	1.351351
1999-11-03	ATVI	1.754386
1999-11-04	ATVI	0.827586
1999-11-05	ATVI	5.198358

In [44]: `data["Ticker"].unique()`

```
Out[44]: array(['ATVI', 'ADBE', 'AMD', 'ALXN', 'ALGN', 'GOOG', 'AMZN', 'AAL',
               'AMGN', 'ADI', 'AAPL', 'AMAT', 'ASML', 'ADSK', 'ADP', 'BIDU',
               'BIIB', 'BMRN', 'BKNG', 'AVGO', 'CDNS', 'CELG', 'CERN', 'CHTR',
               'CHKP', 'CTAS', 'CSCO', 'CTXS', 'CTSH', 'CMCSA', 'COST', 'CSX',
               'CTRP', 'DLTR', 'EBAY', 'EA', 'EXPE', 'FB', 'FAST', 'FISV', 'FO
               X',
               'GILD', 'HAS', 'HSIC', 'IDXX', 'ILMN', 'INCY', 'INTC', 'INTU',
               'ISRG', 'JBHT', 'JD', 'KLAC', 'LRCX', 'LBTYA', 'LULU', 'MAR',
               'MXIM', 'MELI', 'MCHP', 'MU', 'MSFT', 'MDLZ', 'MNST', 'NTAP',
               'NTES', 'NFLX', 'NVDA', 'NXPI', 'ORLY', 'PCAR', 'PAYX', 'PYPL',
               'PEP', 'QCOM', 'REGN', 'ROST', 'SIRI', 'SWKS', 'SBUX', 'SYMC',
               'SNPS', 'TTWO', 'TSLA', 'TXN', 'KHC', 'TMUS', 'ULTA', 'UAL',
               'VRSN', 'VRSK', 'VRTX', 'WBA', 'WDC', 'WLTW', 'WDAY', 'WYNN',
               'XEL', 'XLNX'], dtype=object)
```

```
In [20]: data_tsla = data[data["Ticker"] == "TSLA"]
```

```
In [21]: data_tsla = data_tsla.sort_index().reset_index()
```

```
In [22]: data_tsla.head()
```

```
Out[22]:
```

	Time	Ticker	Close
0	2010-06-29	TSLA	NaN
1	2010-06-30	TSLA	-0.251151
2	2010-07-01	TSLA	-7.847251
3	2010-07-02	TSLA	-12.568306
4	2010-07-06	TSLA	-16.093750

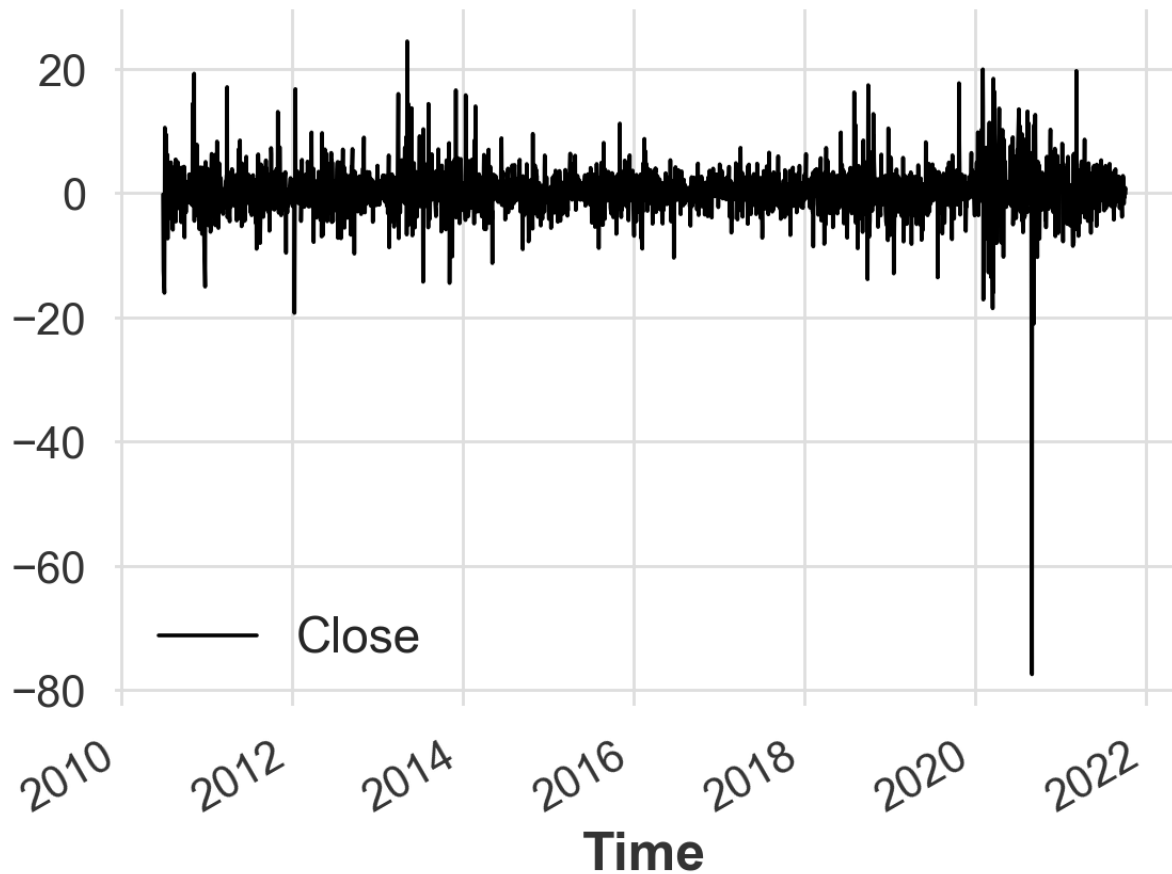
```
In [46]: data_tsla.tail()
```

```
Out[46]:
```

	Time	Ticker	Close
2833	2021-09-29	TSLA	0.482278
2834	2021-09-30	TSLA	-0.746183
2835	2021-10-01	TSLA	-0.033528
2836	2021-10-04	TSLA	0.813962
2837	2021-10-05	TSLA	-0.120277

```
In [48]: data_tsla.plot(x="Time", y="Close")
```

```
Out[48]: <Axes: xlabel='Time'>
```



```
In [23]: from tsfresh import extract_features, select_features
         from tsfresh.utilities.dataframe_functions import roll_time_series, make_
         from tsfresh.utilities.dataframe_functions import impute
```

```
In [24]: data_rolled = roll_time_series(
         data_tsla,
         column_id="Ticker",
         column_sort="Time",
         max_timeshift=20,
         min_timeshift=5,
         )
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/tsfresh/u
tilities/dataframe_functions.py:520: UserWarning: Your time stamps are not
uniformly sampled, which makes rolling nonsensical in some domains.
  warnings.warn(
Rolling: 100%|████████████████████████████████████████| 30/30 [00:02<00:00, 13.2
6it/s]
```

```
In [25]: data_rolled.head()
```

Out[25]:

	Time	Ticker	Close	id
1995	2010-06-29	TSLA	NaN	(TSLA, 2010-07-07 00:00:00)
1996	2010-06-30	TSLA	-0.251151	(TSLA, 2010-07-07 00:00:00)
1997	2010-07-01	TSLA	-7.847251	(TSLA, 2010-07-07 00:00:00)
1998	2010-07-02	TSLA	-12.568306	(TSLA, 2010-07-07 00:00:00)
1999	2010-07-06	TSLA	-16.093750	(TSLA, 2010-07-07 00:00:00)

In [51]: `data_rolled[(data_rolled["Time"] == pd.Timestamp("20100707"))]`

Out[51]:

	Time	Ticker	Close	id
2000	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-07 00:00:00)
2006	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-08 00:00:00)
2013	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-09 00:00:00)
2021	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-12 00:00:00)
2030	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-13 00:00:00)
2040	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-14 00:00:00)
2051	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-15 00:00:00)
2063	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-16 00:00:00)
2076	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-19 00:00:00)
2090	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-20 00:00:00)
2105	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-21 00:00:00)
2121	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-22 00:00:00)
2138	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-23 00:00:00)
2156	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-26 00:00:00)
2175	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-27 00:00:00)
2195	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-28 00:00:00)
2215	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-29 00:00:00)
2235	2010-07-07	TSLA	-1.924271	(TSLA, 2010-07-30 00:00:00)
2255	2010-07-07	TSLA	-1.924271	(TSLA, 2010-08-02 00:00:00)
2275	2010-07-07	TSLA	-1.924271	(TSLA, 2010-08-03 00:00:00)
2295	2010-07-07	TSLA	-1.924271	(TSLA, 2010-08-04 00:00:00)

In [26]: `X = extract_features(`

```
data_rolled.drop("Ticker", axis=1).fillna(0),
column_id="id",
column_sort="Time",
column_value="Close",
impute_function=impute,
show_warnings=False
)
```

Feature Extraction: 100%|████████████████████| 30/30 [00:08<00:00, 3.37it/s]

In [27]: `X.head()`

Out[27]:

		Close__variance_larger_than_standard_deviation	Close__has_duplicat
TSLA	2010-07-07	1.0	
	2010-07-08	1.0	
	2010-07-09	1.0	
	2010-07-12	1.0	
	2010-07-13	1.0	

5 rows x 783 columns

In [28]: `X.shape`

Out[28]: (2833, 783)

In [29]: `X = X.set_index(X.index.map(lambda x: x[1]), drop=True)`
`X.index.name = "last_date"`
`X.head()`

```
Out[29]:
```

last_date	Close__variance_larger_than_standard_deviation	Close__has_duplicate_m
2010-07-07	1.0	(
2010-07-08	1.0	(
2010-07-09	1.0	(
2010-07-12	1.0	(
2010-07-13	1.0	(

5 rows × 783 columns

```
In [30]: y = data_tsla.set_index("Time").sort_index()['Close'].shift(-1).dropna()
```

```
In [31]: y_dummy = data_tsla.set_index("Time").sort_index()['Close'].diff().transf
```

```
In [54]: y_dummy
```

```
Out[54]: 2010-07-07    1.0
2010-07-08    1.0
2010-07-09   -1.0
2010-07-12   -1.0
2010-07-13    1.0
...
2021-09-28   -1.0
2021-09-29    1.0
2021-09-30   -1.0
2021-10-01    1.0
2021-10-04    1.0
Name: Close, Length: 2832, dtype: float64
```

```
In [32]: common_index = X.index.intersection(y.index)
X = X.loc[common_index]
y = y.loc[common_index]
y_dummy = y_dummy.loc[common_index]
```

```
In [33]: X_train = X[:"2018"]
X_test = X["2019":]

y_train = y[:"2018"]
y_test = y["2019":]
y_dummy_train = y_dummy[:"2018"]
```

```
In [55]: X_train_selected = select_features(X_train, y_dummy_train, ml_task="class
```

```
In [57]: X_test_selected = X_test[X_train_selected.columns]
```

```
In [56]: X_train_selected.shape
```

```
Out[56]: (2137, 57)
```

```
In [37]: from xgboost import XGBRegressor
```

```
In [38]: xgb = XGBRegressor(n_estimators=100, max_depth=3, )
```

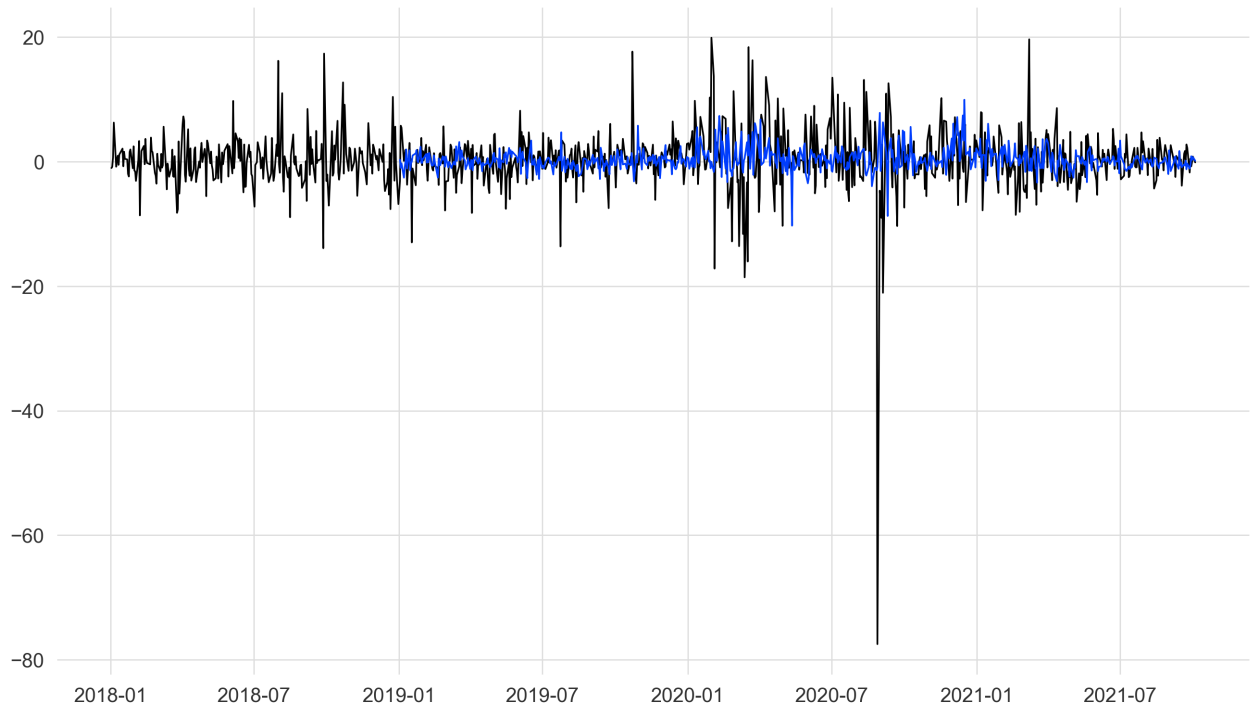
```
In [39]: xgb.fit(X_train_selected, y_train)
```

```
Out[39]: XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_r
ounds=None,
              enable_categorical=False, eval_metric=None, feature_
types=None,
              gamma=None, grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, ma
x_bin=None,
```

```
In [40]: xgb_pred = pd.Series(xgb.predict(X_test_selected), index=X_test_selected.
```

```
In [41]: fig, ax = plt.subplots(figsize=(16,9))
ax.plot(y["2018:"], label="Ground Truth")
ax.plot(xgb_pred)
```

```
Out[41]: [<matplotlib.lines.Line2D at 0x1586cbad0>]
```



```
In [58]: from sklearn.metrics import mean_squared_error, mean_absolute_percentage_
```

```
In [43]: mean_squared_error(y_test, xgb_pred)
```

```
Out[43]: 28.908068725603076
```

```
In [62]: mean_squared_error(y_test, np.ones_like(y_test) * y_test.mean())
```

```
Out[62]: 26.551338195389704
```

```
In [71]: mean_squared_error(y_test, y_test.shift(1).fillna(0))
```

```
Out[71]: 51.086028269445585
```